GUIDE · EN / ES

Git Visual Lab

A visual, hands-on guide to Git

Git is a time machine for your project. It saves snapshots (commits), lets you work on ideas in parallel (branches) and combine them safely (merge and rebase). This guide follows the 10 levels of the interactive app: each one has a short lesson, the starting graph, the commands that solve it and the result.

Play it online: `davidcostacv.github.io/git-visual-lab/`

How to read the graphs: circles are commits (older on the left). Rounded labels are branches; the filled one is where HEAD is. Dashed labels are tags. Faded commits are not reachable anymore.

LEVEL 1 / 10

Commits: save points

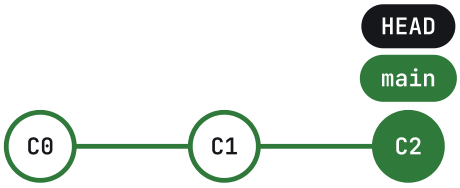
A **commit** is a snapshot of your whole project at one moment, like a save point in a video game. Each new commit remembers the one before it (its *parent*), so together they form a history you can always go back to.

Each circle in the graph is a commit. **main** is the name of your branch, and **HEAD** marks where you are standing right now.

BEFORE



AFTER



COMMANDS

```
$ git commit -m "Add homepage"
$ git commit -m "Add contact form"
```

In real life: In a real project you first run `git add file` to choose what goes in the snapshot, then `git commit -m "what I did"`. A good message explains *why*.

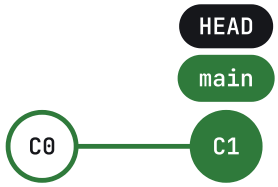
LEVEL 2 / 10

Branches: parallel worlds

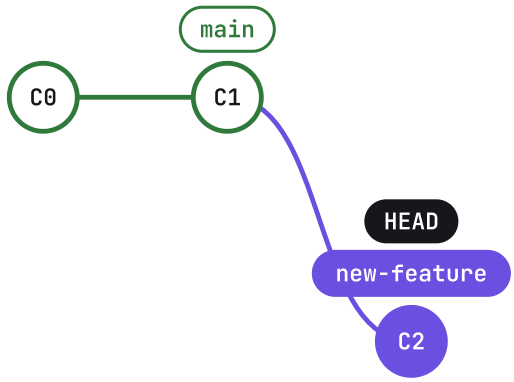
A **branch** is just a sticky note with a name, pointing at a commit. Creating one is instant and costs nothing.

Branches let you try an idea without touching **main**. When you commit, only the branch you are on (the one HEAD is attached to) moves forward.

BEFORE



AFTER



COMMANDS

```
$ git switch -c new-feature
$ git commit
```

In real life: Teams create one branch per feature or bug fix, for example `feature/login`. `git switch -c name` creates the branch and moves you onto it in one step.

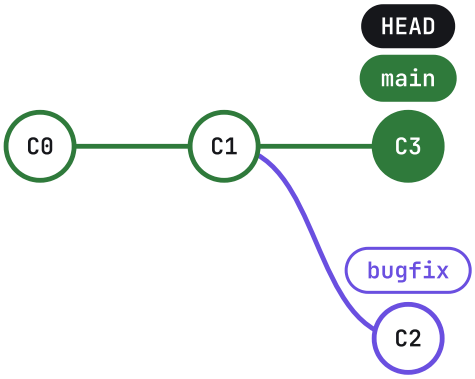
LEVEL 3 / 10

Merge: bring work together

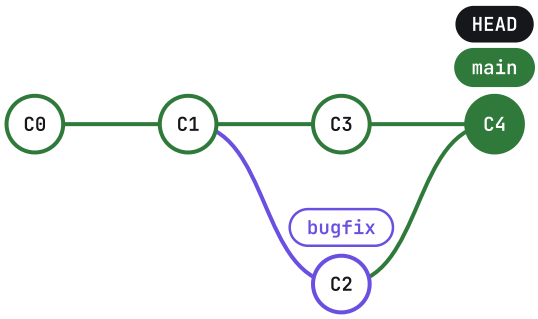
Your fix lives on **bugfix**, but users only get what is on **main**. **git merge** joins two histories.

Because both branches moved, Git creates a **merge commit**: a commit with *two parents*. Now main contains everything from both sides.

BEFORE



AFTER



COMMANDS

```
$ git merge bugfix
```

In real life: Stand on the branch that *receives* the work (usually main), then merge the other one into it. On GitHub, the "Merge pull request" button does exactly this.

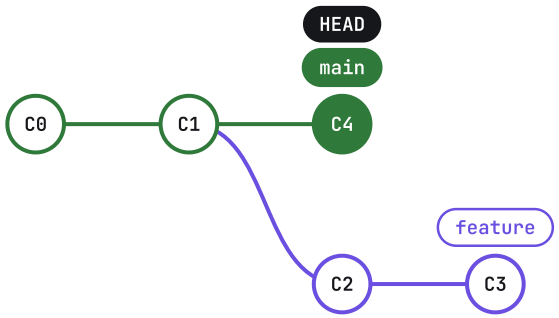
LEVEL 4 / 10

Rebase: a straight line

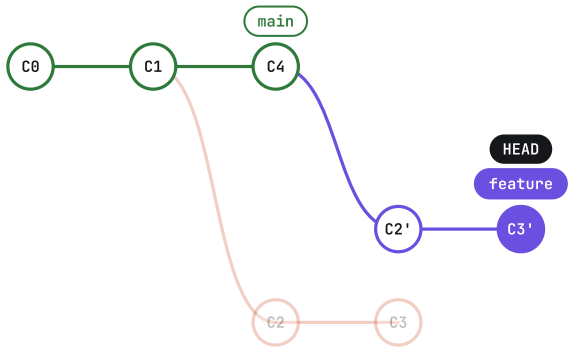
Rebase is the other way to combine work. Instead of a merge commit, it *copies* your commits and replays them on top of another branch, as if you had started your work today.

Copies get a prime: **C2** becomes **C2'**. The old ones fade away because nothing points at them anymore.

BEFORE



AFTER



COMMANDS

```
$ git switch feature
$ git rebase main
```

In real life: Use rebase to update *your own* branch before opening a pull request. Golden rule: never rebase commits other people already pulled, because it rewrites history.

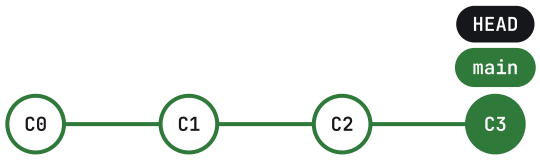
LEVEL 5 / 10

HEAD: where you stand

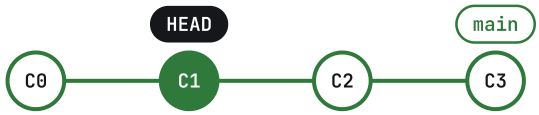
HEAD means "you are here". Normally it is attached to a branch. If you check out a *commit* instead of a branch, HEAD becomes **detached**: you are visiting the past.

Every commit has an id (C0, C1…). In real Git it looks like `a1b2c3d`.

BEFORE



AFTER



COMMANDS

```
$ git checkout C1
```

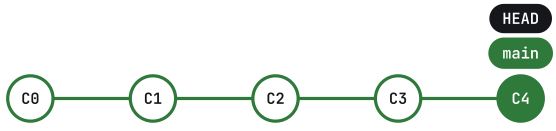
In real life: Detached HEAD is great for looking at old code or finding when a bug appeared. If you want to keep work you do there, create a branch: `git switch -c rescue`.

LEVEL 6 / 10

Relative refs: ~ and ^

Typing ids is slow. Git lets you move relative to a name: **main^** is "the parent of main", and **main~3** is "3 steps back from main". **HEAD~1** is "one step back from where I am".

BEFORE



AFTER



COMMANDS

```
$ git branch review main~2
$ git checkout main^
```

In real life: `git diff HEAD~1` shows what changed in the last commit. `git branch -f name ref` moves a branch anywhere, which is handy to fix a branch you created in the wrong spot.

LEVEL 7 / 10

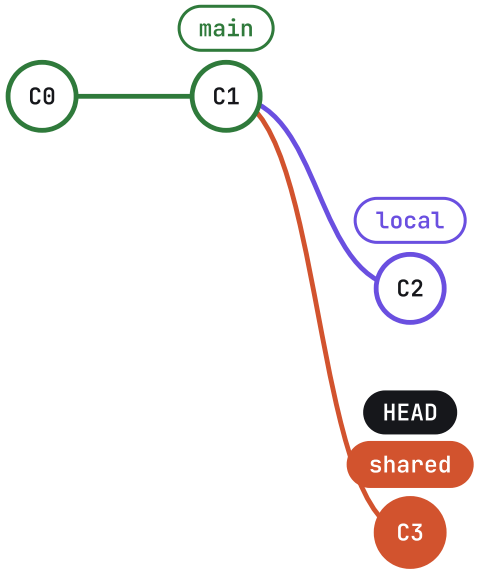
Undo: reset vs revert

Two ways to undo a bad commit:

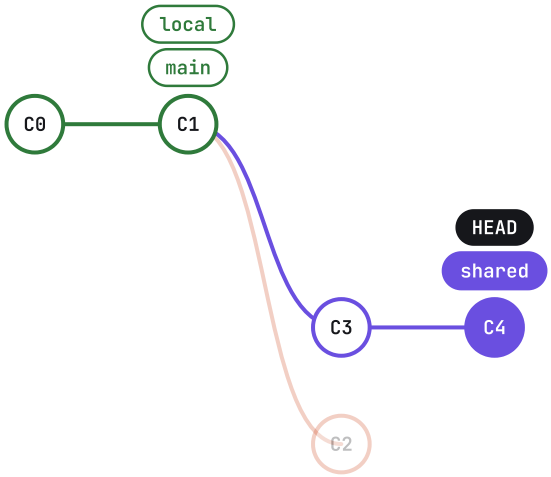
git reset moves the branch backwards, as if the commit never happened. Use it only on *private* work.

git revert adds a *new* commit that cancels the bad one. History stays honest, so it is safe on *shared* branches.

BEFORE



AFTER



COMMANDS

```
$ git switch local
$ git reset HEAD~1
$ git switch shared
$ git revert HEAD
```

In real life: Rule of thumb: already pushed? Revert. Only on your computer? Reset is fine. Real reset also has modes: `--soft` keeps your changes, `--hard` throws them away.

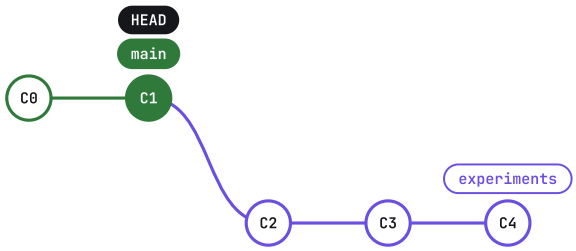
LEVEL 8 / 10

Cherry-pick: choose commits

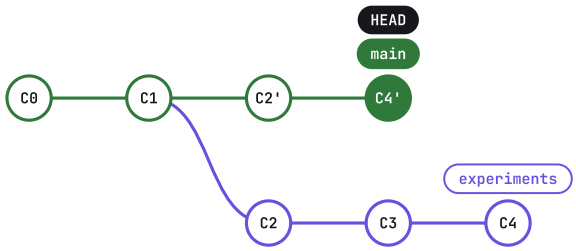
Sometimes you want *some* commits from a branch, not all of them. **git cherry-pick** copies exactly the commits you name onto where you are.

The **experiments** branch has two useful fixes (C2 and C4) and one weird test (C3) you do not want.

BEFORE



AFTER



COMMANDS

```
$ git cherry-pick C2 C4
```

In real life: Common use: a bug is fixed on a development branch and you need just that fix in the version already released.

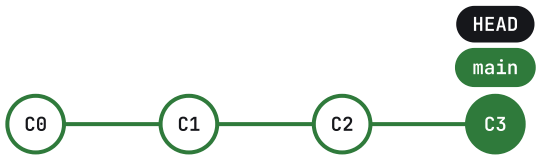
LEVEL 9 / 10

Tags: mark versions

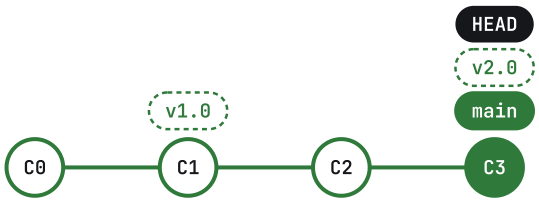
A **tag** is a sticky note that never moves. Branches move forward with every commit; tags stay put forever.

That is why tags are used for releases: **v1.0** will always mean exactly that snapshot.

BEFORE



AFTER



COMMANDS

```
$ git tag v1.0 C1
$ git tag v2.0
```

In real life: GitHub turns tags into Releases, where you can attach files (like the PDF of this guide). Push them with `git push --tags`.

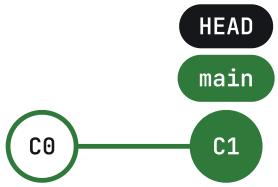
LEVEL 10 / 10

Final challenge: a real workflow

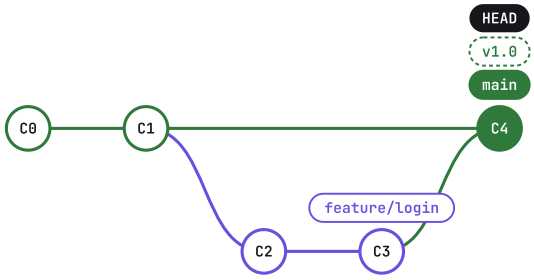
Put it all together, the way teams work every day:

1. Create **feature/login** and make 2 commits there.
2. Go back to **main** and merge it with `--no-ff`, so the history shows that a feature was merged (even if a fast-forward was possible).
3. Tag the result as **v1.0**.

BEFORE



AFTER



COMMANDS

```
$ git switch -c feature/login
$ git commit -m "Login form"
$ git commit -m "Login validation"
$ git switch main
$ git merge --no-ff feature/login
$ git tag v1.0
```

In real life: This is exactly how this project was built on GitHub: one branch per feature, a pull request for each one, merged into main, then tagged as a release.

Cheat sheet

`git commit -m "msg"`	Save a snapshot
`git switch -c name`	Create a branch and move to it
`git switch name`	Move to a branch
`git merge name`	Join another branch into yours
`git rebase name`	Replay your commits on top of another branch
`git checkout C1`	Visit an old commit (detached HEAD)
`main~2 / HEAD^`	Two commits back / the parent
`git reset HEAD~1`	Move the branch back (private work only)
`git revert C3`	Add a commit that undoes C3 (safe when shared)
`git cherry-pick C2 C4`	Copy only those commits here
`git tag v1.0`	Mark a version (tags never move)
`git log / git status`	See history / where you are